ЛАБОРАТОРНА РОБОТА № 6
ВІЛЬНА ТЕМА

Мета роботи: змоделювати головну проблему наступного року навчання (у вигляді вибору теми дипломного проекту) шляхом самостійного формування завдання на 6 лабораторну роботу.

Завдання

Розробити мобільний застосунок під платформу Android у вигляді 2D гри з нескінченним ігровим процесом.

У межах виконання роботи необхідно реалізувати:

- створення користувацького графічного інтерфейсу за допомогою кастомного компонента View;
- відображення ігрових об'єктів (персонажа та платформ) на екрані пристрою;
- реалізацію фізичної моделі руху персонажа з урахуванням гравітації та початкової швидкості;
- обробку взаємодії користувача з грою через сенсорний екран (керування рухом персонажа);
- алгоритм генерації платформ із використанням випадкових величин для забезпечення варіативності ігрового процесу;
- перевірку зіткнень персонажа з платформами та відповідну зміну його руху;
- реалізацію безперервного ігрового процесу (нескінченний рівень) шляхом динамічного оновлення положення об'єктів;
- підрахунок та відображення кількості набраних очок;
- обробку ситуації завершення гри (падіння персонажа) з подальшим перезапуском ігрового процесу.

Реалізація

У процесі виконання лабораторної роботи було розроблено мобільний застосунок для платформи Android у вигляді 2D гри з нескінченним ігровим процесом.

Основна логіка гри реалізована у власному класі GameView, який успадковує стандартний компонент View. Це дозволило повністю контролювати процес відображення графіки та оновлення ігрового стану.

Після створення застосунку було налаштовано головну активність (MainActivity), яка підключає розроблений ігровий компонент через XML-розмітку. У файлі activity_main.xml використано контейнер FrameLayout, у якому розміщено GameView на весь екран пристрою.

Ігровий процес організовано у методі onDraw(), який відповідає за відмалювання всіх об'єктів на екрані. У цьому методі відбувається очищення екрану, відображення персонажа у вигляді прямокутника, відображення платформ, а також виведення поточного рахунку. Після кожного кадру викликається метод update(), який оновлює стан гри, та invalidate(), що забезпечує безперервне перемальовування екрану.

Фізика руху персонажа реалізована за допомогою змінних швидкості та гравітації. На кожному кроці швидкість змінюється під впливом гравітації, після чого оновлюється позиція персонажа по вертикалі.

Керування персонажем здійснюється через обробку дотиків до екрану. У методі onTouchEvent() визначається, у якій частині екрану відбувся дотик, і відповідно змінюється положення персонажа по горизонталі (вліво або вправо).

Платформи зберігаються у вигляді списку координат. Початкове створення платформ виконується у методі onSizeChanged(), де вони генеруються з урахуванням допустимих відстаней по горизонталі та вертикалі. Для забезпечення варіативності використовується генерація випадкових значень.

Під час гри реалізовано механізм безперервного руху: коли персонаж піднімається вище середини екрану, всі платформи зміщуються вниз, що створює ефект руху камери. Одночасно з цим збільшується рахунок гравця.

Обробка зіткнень реалізована шляхом перевірки перетину координат персонажа та платформ. У разі контакту і при русі вниз швидкість персонажа змінюється на протилежну, що імітує стрибок.

Для підтримки нескінченного ігрового процесу реалізовано динамічну генерацію платформ. Якщо платформа виходить за межі екрану, вона переміщується у верхню частину з новими випадковими координатами.

У випадку, якщо персонаж падає нижче екрану, відбувається скидання позиції, швидкості та рахунку, що означає початок нової гри.

Таким чином, у роботі було реалізовано повноцінний ігровий цикл, який включає відображення, оновлення стану, обробку взаємодії користувача та генерацію ігрових об'єктів.

Тестування

Після завершення реалізації мобільного застосунку було проведено його тестування з метою перевірки коректності роботи основних функцій, стабільності ігрового процесу та взаємодії користувача з системою.

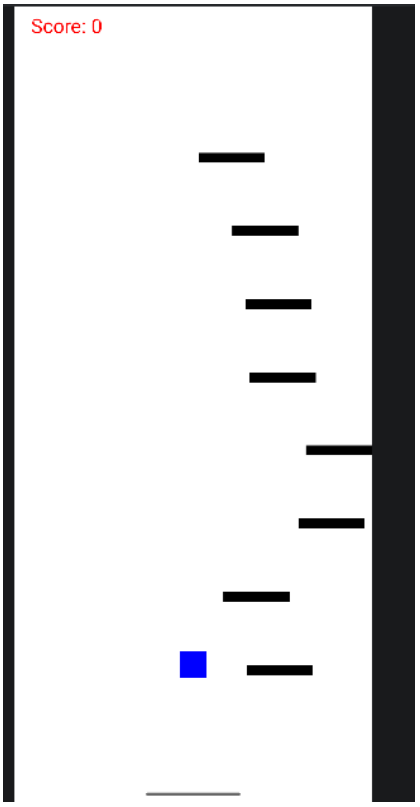

Рисунок 1 — Початковий стан гри після запуску застосунку

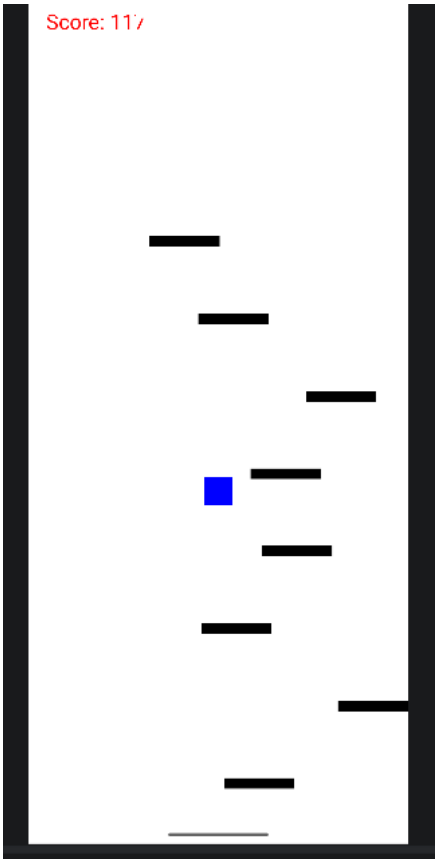

Рисунок 2 — Процес стрибка персонажа після зіткнення з платформою

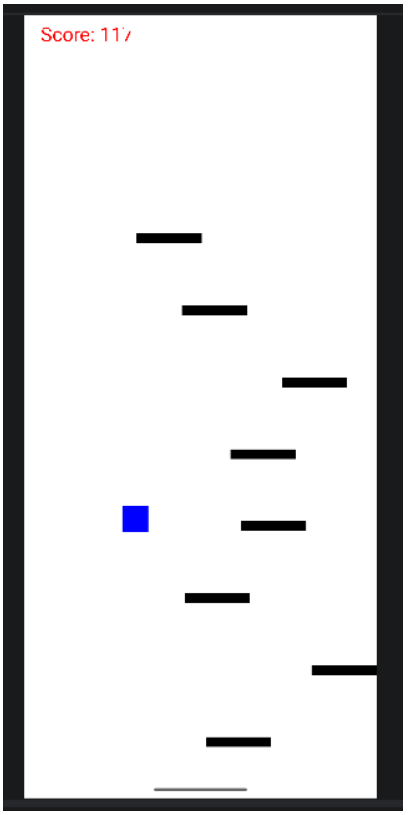

Рисунок 3 — Відображення руху персонажа вліво/вправо

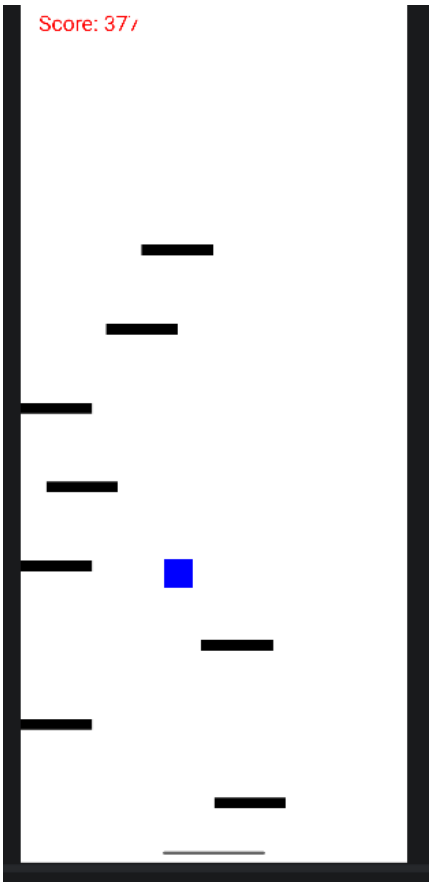

Рисунок 4 — Збільшення рахунку під час підйому персонажа

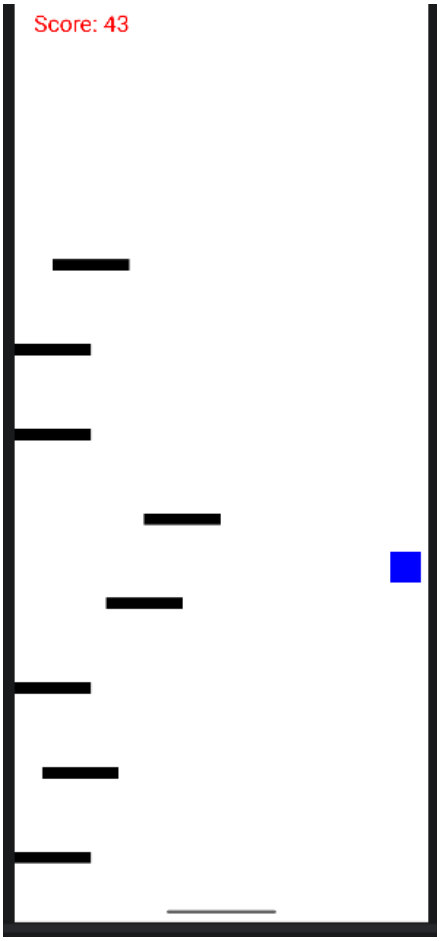

Рисунок 5 — Перехід персонажа через межі екрану

Висновки

У процесі виконання лабораторної роботи було розроблено мобільний застосунок для платформи Android у вигляді 2D гри з нескінченним ігровим процесом.

Було самостійно сформульовано задачу, обрано тему, яка не розглядалася у попередніх лабораторних роботах, та реалізовано програмний продукт із використанням кастомного графічного компонента.

У ході роботи реалізовано основні елементи ігрової логіки, зокрема фізику руху персонажа з урахуванням гравітації, обробку зіткнень із платформами, генерацію ігрових об'єктів із використанням випадкових величин, а також механізм безперервного ігрового процесу.

Також було реалізовано взаємодію користувача із застосунком через сенсорний екран, систему підрахунку очок та обробку завершення гри з можливістю її перезапуску.

У результаті виконання роботи було закріплено практичні навички розробки Android-застосунків, роботи з графікою, обробки подій та створення інтерактивних систем.

Отриманий програмний продукт може бути розширений та вдосконалений, що дозволяє розглядати його як основу для подальшої розробки, зокрема у межах дипломного проєкту.

Репозиторій GitHub:
https://github.com/IvanYefremovGit/lab6

Код програми:

MainActivity.kt:

```
package com.example.lab6

import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)
   }
}
```

GameView.kt:

```
package com.example.lab6

import android.content.Context
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.util.AttributeSet
import android.view.MotionEvent
import android.view.View
import kotlin.random.Random
```

```
class GameView(context: Context, attrs: AttributeSet) :
View(context, attrs) {

   private val paint = Paint()

   private var playerX = 0f
   private var playerY = 0f
   private var velocityY = -20f

   private val gravity = 1f

   private val platforms = mutableListOf<Pair<Float, Float>>()

   private val MAX_GAP_Y = 220f
   private val MAX_GAP_X = 220f

   private var score = 0

   override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh:
Int) {
       super.onSizeChanged(w, h, oldw, oldh)

       playerX = w / 2f - 40
       playerY = h - 300f

       platforms.clear()

       var lastY = h - 200f

       for (i in 0..7) {

           val baseX = if (platforms.isEmpty()) {
               w / 2f
           } else {
               platforms.random().first
           }

           val offset = Random.nextFloat() * MAX_GAP_X * 2 -
MAX_GAP_X
           var newX = baseX + offset

           if (newX < 0) newX = 0f
           if (newX > w - 200) newX = w - 200f

           val newY = lastY - MAX_GAP_Y

           platforms.add(Pair(newX, newY))
           lastY = newY
```

```
        }
    }

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)

        canvas.drawColor(Color.WHITE)

        paint.color = Color.BLUE
        canvas.drawRect(playerX, playerY, playerX + 80, playerY +
80, paint)

        paint.color = Color.BLACK
        for (platform in platforms) {
            canvas.drawRect(
                platform.first,
                platform.second,
                platform.first + 200,
                platform.second + 30,
                paint
            )
        }

        paint.color = Color.RED
        paint.textSize = 60f
        canvas.drawText("Score: $score", 50f, 80f, paint)

        update()
        invalidate()
    }

    private fun update() {
        velocityY += gravity
        playerY += velocityY

        if (playerX < -80) {
            playerX = width.toFloat()
        } else if (playerX > width) {
            playerX = -80f
        }

        if (playerY < height / 2) {
            val diff = height / 2 - playerY
            playerY = height / 2f

            score += (diff / 10).toInt()

            for (i in platforms.indices) {
```

```
                val (x, y) = platforms[i]
                platforms[i] = Pair(x, y + diff)
            }
        }

        for (platform in platforms) {
            if (playerY + 80 >= platform.second &&
                playerY + 80 <= platform.second + 30 &&
                playerX + 80 >= platform.first &&
                playerX <= platform.first + 200 &&
                velocityY > 0
            ) {
                velocityY = -32f
            }
        }

        for (i in platforms.indices) {
            val (x, y) = platforms[i]

            if (y > height) {

                var minY = Float.MAX_VALUE
                for (p in platforms) {
                    if (p.second < minY) {
                        minY = p.second
                    }
                }

                val baseX = platforms.random().first

                val offset = Random.nextFloat() * MAX_GAP_X * 2 -
MAX_GAP_X
                var newX = baseX + offset

                if (newX < 0) newX = 0f
                if (newX > width - 200) newX = width - 200f

                val newY = minY - MAX_GAP_Y

                platforms[i] = Pair(newX, newY)
            }
        }

        if (playerY > height) {
            playerX = width / 2f - 40
            playerY = height - 300f
            velocityY = -20f
            score = 0
```

```
        }
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
        if (event.action == MotionEvent.ACTION_DOWN) {
            if (event.x < width / 2) {
                playerX -= 100
            } else {
                playerX += 100
            }
        }
        return true
    }
}
```

activity_main.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.example.lab6.GameView
        android:id="@+id/gameView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"/>

</FrameLayout>
```